PA194NQ: a different 194-state PA-consistency machine

A current-corpus comparison, a commuting source construction, and a refutation of a tempting 193-state shortcut

Research report prepared for Vladimir Reshetnikov

13 November 2026, Pacific time

Abstract

We compare the 51 delivered PA-machine research packets present in `VladimirReshetnikov/ProveIt` at commit `3cc7fbf75ba111a0da696ef2f6001dd528891bd5`. The recognized 194-state frontier has 34 distinct control graphs: 21 accompanied by reported ordinary PA-calibration arguments, nine auxiliary or search designs, and four rejected designs. Several headline names are duplicates.

No machine below 194 states with the required consistency equivalence is established. The accepted alternative is an explicit new 194-state binary, one-tape, blank-start machine, PA194NQ. It combines no-push modus ponens with the exchanged quantifier-distribution emitter. These changes commute at the resolved source level. They alter recurring numerical and storage behavior; the resulting table is absent from all 34 prior graph classes, even after state renaming, root changes, reflection, and symbol exchange.

Using the supplied ordinary syntactic calibration, we give a PA-formalizable proof of

$$\mathsf{PA} \vdash \neg\,\mathrm{Halt}(\mathrm{PA194NQ}) \leftrightarrow \mathrm{Con}(\mathsf{PA}).$$

Both physical checkers pass all 4,755 primitive source outcomes, and an independent checker rederives the four register-list lemmas from the literal table. The complete blank startup is checked at 58,154 transitions. The proof is an ordinary mathematical proof with executable certificates, not a newly compiled end-to-end Lean or Rocq theorem.

A second result explains why a physically valid 193-state coefficient-three candidate is unsuitable: its numerical calculus has an explicit fourteen-line refutation, also expanded into 73,393 independently checked primitive steps. This failure occurs at the logical layer, despite successful source and startup checks.

Contents

1 Result, conventions, and proof boundary **3**
1.1 The accepted literal machine . . . 3
1.2 What is and is not new . . . 3

2 Comparison of the current corpus **3**
2.1 Pinned scope and deduplication . . . 3
2.2 Important exact duplicates . . . 4
2.3 Mechanism-level comparison . . . 4

3 The numerical calculus and the exchanged head **5**

4 Commands, carried stores, and proof compilation **6**
4.1 Exact contracts . . . 6
4.2 Uniform stack words . . . 7
4.3 Axiom selectors and generalization . . . 7
4.4 The missing zero is reserved by the proof compiler . . . 7
4.5 Soundness and the fair scheduler . . . 8

5 Source construction and literal tape verification **8**
5.1 Two changes commute . . . 8
5.2 Allocation and simultaneous folds . . . 9
5.3 What the universal checks establish . . . 9

6 Consistency calibration and inherited dependencies **10**
6.1 PA proofs produced from numerical derivations . . . 10
6.2 Numerical derivations produced from PA refutations . . . 10
6.3 Dependency ledger . . . 10

7 Why the alternative is materially different **11**

8 Attempts below 194 and a rejected 193-state machine **11**
8.1 Protected-runtime quotient search . . . 11
8.2 Other investigated changes . . . 12
8.3 A literal 193-state shortcut with the wrong logic . . . 12
8.4 A concrete obstruction to the three-residue shortcut . . . 12

9 Reproduction and artifact map **14**

A Complete accepted transition table **15**

B Pinned primary sources **16**

1 Result, conventions, and proof boundary

1.1 The accepted literal machine

The authoritative table is `machine/machine.tm`. Its working states are $0, \ldots, 193$, its initial state is 0, and its separate terminal halt label is 194. The halt label has no outgoing rows and is excluded from the state count. The tape is two-way infinite over $\{0, 1\}$, initially all zero, with the head at position zero. Every transition writes one bit and moves exactly one cell left or right. There are 388 transitions.

Each line has the format

`current_state read_bit write_bit L/R next_state.`

The table's SHA-256 is

`d1cf0f6e7a3c8f999d045694efc107dd1f9a03d45dd6c15e3d197ff7754c7231.`

The same machine in sparse compiler labels is `machine/machine.native.tm`, with initial label 74 and halt label 254. `machine/labels.json` gives the exact bijection from dense labels to native labels, including halt. Every working state is graph-reachable from the designated start; this is a graph fact, not a claim that each control is visited on the actual blank run.

Theorem 1.1 (Ordinary mathematical PA calibration)**.** *For the exact table specified above,*

$$\mathsf{PA} \vdash \big(\neg\,\mathrm{Halt}(\mathrm{PA194NQ}, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathsf{PA})\big).$$

Here halting is the usual existence of a coded finite run from the stated blank configuration, and $\mathrm{Con}(\mathsf{PA})$ *means that no finite PA proof derives* $0 = 1$*, for the same ordinary effective presentation of PA used in the inherited calibration.*

The theorem is proved by finite syntactic transformations and source/tape simulation below. Its logical calibration depends on identified supplied mathematical results, whose texts are retained in `references/logical/`. We do not turn a conditional Lean theorem into an unconditional one, nor infer PA derivability from a standard-model execution experiment.

1.2 What is and is not new

The separate-head numerical calculus, no-push MP rule, premise-exchange theorem, compiler, and physical proof engines are supplied prior work [2, 3, 4, 5]. The new accepted object is their particular 194-state composition, with exact source commutation, a complete preservation argument, an updated full-corpus comparison, and fresh independent physical checks. This is not a claim of a new axiom basis, a state-count breakthrough, or global minimality.

The current repository pursues the stronger endpoint $\mathrm{iPRA} \vdash \mathrm{Con}(\mathsf{PA}) \leftrightarrow \neg\,\mathrm{Halt}(M)$, and explicitly states that neither machine-specific direction is complete for its primary PA194 endpoint [1]. That limitation is respected here. An ordinary proof about PA derivability is a different result from an end-to-end proof-assistant export or a proof in iPRA. The Python certificates expose their trusted mathematical rules; they do not constitute a foundational kernel.

2 Comparison of the current corpus

2.1 Pinned scope and deduplication

The comparison is pinned to commit `3cc7fbf75ba111a0da696ef2f6001dd528891bd5`, using both requested PA and incoming paths. The repository had already moved 47 archives into `Computability/BusyBeaver/Research/PA/docs`; four newer ZIPs remain in `docs/incoming`. The packet ledger therefore covers 51 deliveries.

The audit reads the pinned Git object tree, scans all 17,557 tracked blobs under the two primary paths, and recursively inspects ZIP payloads, expanding identical payloads once (30 archive paths, 18 distinct payloads). It recognizes complete five-column numeric binary tables in `.tm`, `.txt`, `.tsv`, and `.csv` files. It does not pretend to recognize every transition system encoded only in JSON, program source, or prose. Source-only proposals are separately considered in the narrative prior-art audit. The contribution claims of all packet READMEs are reviewed; the historical proofs are not all independently rerun.

Quantity	Count
Filed packets / fresh incoming ZIPs	47 / 4
Recognized current literal table occurrences	1,811
Distinct current byte forms	683
Action-graph classes with a spanning root	487
Additional disconnected synthetic fixtures	2
Current 194-state graph classes	34
Reported ordinary PA arguments / auxiliary / rejected	21 / 9 / 4

These are not counts of proved consistency machines. The scan includes unfolded parents, rejected experiments, search inputs, and reference copies. The repository's older catalog describes 3,803 originally delivered table paths; many redundant physical files have since been removed. Its count therefore differs from the current literal-occurrence count. The complete machine and packet records, including exact paths and hashes, are in `comparison/frontier194.csv` and `comparison/packet_ledger.csv`.

There are 26 194-state classes with a start declared in the table or fixed by the dense convention. Eight other classes occur only in native numbering without a start declaration in that file. For the cosmetic-novelty exclusion, the comparison conservatively allows every graph-spanning root instead of silently assigning one.

2.2 Important exact duplicates

PA194HS and PA194NS deliver identical dense bytes, with SHA-256 beginning `19935635f2c5`. The two incoming N6 packets deliver identical PA194N6 bytes, beginning `36da42a81d60`. PA194X is PA194CSZ. Five headline PA194C submissions repeat one principal machine. Consequently the four incoming archives add exactly four new 194-state classes: H, HS/NS, N5, and N6. Counting filenames or submitted reports would overstate the frontier.

2.3 Mechanism-level comparison

Family	Contribution and limitation
U208 and PA209	Earlier common ancestry, with conditional-context, entry-context, and finite-history reductions. Useful for the source-calculus and history-fold techniques, but larger than the current frontier.
PA196	James Chang's imported development and alternative decoder layouts. The exported PA implication from literal nonhalting to consistency does not by itself supply its converse. Later ordinary reports add the syntactic reverse direction.
PA195 families	Separate diagonal head, reader reassociation, and compatible folds; several genuinely different 195-state controllers and source emitters also remain. A rejected smaller scanner must not be counted as a result.
PA194, 68/8, AD	The main count is $134 + 67 - 7 = 194$; a second source-identical layout has 68 readers and eight folds. AD exchanges physical register slots.
C, CS, CZ, CSZ/X	Cached-result MP, cache-only Shift, symmetric comparison, and known-zero entry jumps. Some compositions have their own invariant and physical evidence; CZ is retained as auxiliary in the conservative ledger.
M4R, M5, M6	Reassociated constructor implementation and coefficients five or six. These require formula recoding and a generic constructor proof, not just a changed literal numeral.
D and Q	D changes the separately emitted diagonal instance to $\Gamma(D)$. Q exchanges the two curried premises of quantifier distribution. Both need logical proof translations.
J, F, A, R, ZR	Known-zero decrements replace source jumps; R also changes 1,111 source addresses. An invariant and a separate bootstrap check are essential. Some nearby source-valid variants halt during bootstrap.
N	Removes successful MP's dummy-zero push, retains the consequent in visible top and offset, and retains the original $T + H = 1$ Shift test. Its all-store compiler explicitly reserves the missing zero.
H, HS/NS	Compose N-style MP with cache-only Shift, with or without the symmetric comparator. HS and NS are one table.

Family	Contribution and limitation
N5, N6	Compose no-push MP with coefficient-five or coefficient-six constructors; N6 has two duplicate incoming deliveries.
PA194NQ (this report)	Combines N's visible-top MP and original Shift with the exchanged distribution emitter. The 79 changed instructions and 19-site MP deletion occupy disjoint source regions.

The full frontier ledger also lists nine auxiliary and four rejected graph classes individually. Three rejected comparator-return variants halt on blank tape after 1,361 steps according to their supplied exact replays. The fourth rejected design is a shortened source-address scanner. Neither a plausible source rewrite nor valid local folds establish a correct blank machine by themselves.

Source-level prior art includes separate $\Gamma(H)$, fusions of diagonal vacuity into several other heads, distribution and currying inside the separation gate, premise and parameter permutations, and residual MP. We do not claim those proposals as new. The ZFC work contributes reusable history-sensitive folds, whole-register-list reasoning, and carried-store compiler techniques. The concise-iPRA development helps distinguish proof levels, but substituting an iPRA-consistency checker would change the target consistency statement; it is not an automatic state reduction for this task [7].

3 The numerical calculus and the exchanged head

Let

$$p(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \quad I(a,b) = 4p(a,b), \quad M(a,b) = 4p(a,b)+1,$$
$$U(i,B) = 4p(B,i)+2, \qquad E(a,b) = 4p(a,b)+3.$$

Code 0 denotes truth, code 1 is $M(0,0)$, and $N(B) = I(B,1)$ denotes negation. Write $A \to B = I(A,B)$, $A \wedge B = N(I(A,N(B)))$, and $\exists x_i B = N(U(i,N(B)))$. All these expressions denote natural-number operations. In particular, $I(0,0) = 0$; the proof never assumes a freely generated injective formula syntax at that exceptional code.

For arbitrary natural parameters D, A, H, the six ordinary AX heads in emission order are

$$\begin{aligned}
f_1 &= (\exists x_D\, U(D,H)) \to H,\\
f_2 &= U(D,U(A,H)) \to U(A,U(D,H)),\\
f_3 &= M(H,D) \to (E(H,A) \to M(A,D)),\\
f_4 &= E(H,D) \to (M(A,H) \to M(A,D)),\\
f_5 &= \exists x_D E(H,D),\\
f_6 &= U(D,A) \to \big(U(D,I(A,H)) \to U(D,H)\big).
\end{aligned}$$

PA194NQ replaces only the last head by

$$f_6^Q = U(D,I(A,H)) \to \big(U(D,A) \to U(D,H)\big). \tag{1}$$

The separate diagonal head is $\Gamma(A) = I(A,U(A,A))$. The dominated implicational head is

$$\rho(D,A,H,d) = I\big(I(I(H,d),D), I(I(D,H),I(A,H))\big), \qquad d \le D.$$

For completeness, the remaining guarded gate uses

$$\begin{aligned}
V &= M(H,D) \to U(A,M(H,D)), & P &= A \to M(A,D),\\
S &= U\big(D,(M(H,D)\to P)\wedge(M(A,D)\to A)\big),\\
Q_0 &= U(A,S \to N(P)).
\end{aligned}$$

Its value is $g(H,D,A) = N(I(V,Q_0))$ when $A \neq D,H$, and 0 otherwise. Truth, these heads, modus ponens, and generalization of theorems define the supplied separate-head numerical calculus $\mathcal{C}$. Replacing f_6 by f_6^Q defines $\mathcal{C}_Q$.

Lemma 3.1 (Uniform premise exchange)**.** *The calculi $\mathcal{C}$ and $\mathcal{C}_Q$ have primitive-recursive proof translations in both directions preserving every numerical conclusion. PA proves the correctness of these translations.*

Proof. The unchanged restricted ρ basis derives the implicational K and S schemas and hence the theorem

$$(P \to (Q \to R)) \to (Q \to (P \to R)).$$

`mathematics/exchange_certificate.py` constructs and independently replays a 61-line parametric certificate: 17 K instances, 14 S instances, and 30 applications of MP, with no hypotheses. Substituting

$$P = U(D, A), \quad Q = U(D, I(A, H)), \quad R = U(D, H)$$

converts f_6 to f_6^Q. Interchanging P and Q gives the reverse conversion. Replace each affected axiom line by this fixed proof template; all other axioms and inference rules are unchanged. Instantiating a finite template and concatenating coded finite derivations are primitive recursive. Their verification is an induction on the input derivation, formalizable in PA. Since this argument evaluates identities in the numerical implication operation, it does not require disjoint syntax at code zero. □

The compiler's emitter is also checked independently as a free-constructor calculation before numerical evaluation. Its complete word has 190 macro tokens. Only positions 102 through 120, in zero-based numbering, change. The old and new blocks have the same parameter use and stack effect; their final formulas are exactly the two displayed heads. For example, at $(A, D, H) = (1, 0, 0)$, their codes are respectively 31,609,172 and 76,268. This is a recurring numerical difference, not a change of labels.

4 Commands, carried stores, and proof compilation

4.1 Exact contracts

A command store is (t, w, d, h), consisting of visible top, Cantor-coded stack, numerical offset, and cached theorem. Write $[a; w] = p(a, w)$, and use iterated brackets with the leftmost entry on top. The total projections satisfy

$$\mathrm{hd}[a; w] = a, \quad \mathrm{tl}[a; w] = w, \quad [0; 0] = 0, \quad \mathrm{hd}(0) = \mathrm{tl}(0) = 0.$$

The numerical stack is not a free list.

With $y = d + x$, PA194NQ's MP contract is

$$\mathsf{MP}_x(t, w, d, h) = \begin{cases} (y, w, y, 0), & I(t, y) = h, \\ (0, w, y, 0), & I(t, y) \neq h. \end{cases} \tag{2}$$

Successful MP leaves the stack unchanged, instead of inserting a dummy zero. It retains the consequent in both visible top and numerical offset. Shift is the original operation

$$\mathsf{S}(t, w, d, h) = \begin{cases} \mathsf{halt}, & t + h = 1, \\ (h, \mathrm{tl}(w), d, \mathrm{hd}(w)), & t + h \neq 1. \end{cases} \tag{3}$$

Put $D = d + x$, $H = h + y$. The AX operation is

$$\mathsf{AX}_{A,x,y}(t, w, d, h) = \big(g(H, D, A), [q_1, \ldots, q_9, t; w], 0, 0\big), \tag{4}$$

where

$$(q_1, \ldots, q_9) = (\Gamma(A), f_6^Q, f_5, f_4, f_3, f_2, f_1, \rho(D, A, H, d), U(d, h)).$$

Thus nine generated entries and the saved old top are pushed. Heads $q_1, \ldots, q_8$ and the gate are multiples of four; q_9 has residue two and is at least two. None is code one, independently of any consistency assumption. The guard and cleanup suffix are unchanged.

The following proof is an explicit specialization of the supplied N compiler. The exchanged head changes the interpretation of selector q_2, but not any stack count, residue argument, or command word.

4.2 Uniform stack words

Let $A_0 = \mathsf{AX}_{0,0,0}$, and define

$$\mathsf{Push} = A_0; \mathsf{S}^9; \mathsf{MP}_0, \qquad \mathsf{Pop} = A_0; \mathsf{S}^{11}, \qquad \mathsf{Discard} = \mathsf{MP}_0, \qquad \mathsf{Offset}_i = \mathsf{MP}_i.$$

Their exact contracts are

$$\mathsf{Push}(t, w, d, h) = (0, [t; w], 0, 0), \tag{5}$$
$$\mathsf{Pop}(t, [a; w], d, h) = (t, w, 0, a), \tag{6}$$
$$\mathsf{Discard}(t, w, 0, h) = (0, w, 0, 0), \tag{7}$$
$$\mathsf{Offset}_i(0, w, 0, 0) = (0, w, i, 0). \tag{8}$$

Push and Pop are total on all natural stores, including false top code one. To see this, AX followed by nine Shifts gives $(q_8, [t; w], 0, U(d, h))$; ten Shifts give $(U(d, h), w, 0, t)$. All these Shifts are safe by the displayed residues. The eleventh tests $U(d, h) + t \geq 2$, so it also cannot halt. The MP in Push compares an implication with a residue-two universal code and returns truth. In Discard, both MP branches coincide because the proposed consequent is zero. For Offset, $I(0, i) > 0$ when $i > 0$, so comparison with cached zero fails; the case $i = 0$ has the same stated result.

These equalities also cover the collapsed zero-pair cases: Pop from a zero stack returns cached zero, and Push of zero onto zero returns zero stack. The symbolic word checker expressly includes these cases.

4.3 Axiom selectors and generalization

From a store $(t, w, 0, 0)$, selector

$$\mathsf{Discard}; \mathsf{AX}_{A,D,H}; \mathsf{S}^{j+1}; \mathsf{Pop}^{9-j} \qquad (1 \leq j \leq 7)$$

returns $(q_j, w, 0, 0)$. The remaining $9 - j$ items are precisely the unselected tail and the saved zero; the final Pop installs that zero in cache. In particular selector two returns f_6^Q, not the old f_6. For $u \leq a$, the dominated-head selector is

$$\mathsf{Discard}; \mathsf{Offset}_u; \mathsf{AX}_{b,a-u,c}; \mathsf{S}^9; \mathsf{Pop},$$

which returns $(\rho(a, b, c, u), w, 0, 0)$. The gate selector is $\mathsf{Discard}; \mathsf{AX}_{A,D,H}; \mathsf{Pop}^{10}$. Truth uses Discard alone.

Generalization is compiled by

$$\mathsf{Gen}_i = \mathsf{Push}; \mathsf{Offset}_i; \mathsf{S}; \mathsf{AX}_{i,0,0}; \mathsf{S}^{10}.$$

From $(a, w, 0, 0)$ it returns $(U(i, a), w, 0, 0)$. Immediately before AX, the store is $(0, w, i, a)$; the final ten Shifts apply the AX identity above with saved top zero. This word remains total when $a = 1$.

4.4 The missing zero is reserved by the proof compiler

Suppose recursively compiled proofs return a and $I(a, b)$, each preserving its input stack and clearing offset and cache, or halt early. The MP proof word is

$$\mathsf{Discard}; \mathsf{Push}; C(I(a, b)); \mathsf{Push}; C(a); \mathsf{Push}; \mathsf{S}; \mathsf{S}; \mathsf{MP}_b; \mathsf{Pop}. \tag{9}$$

Before the two consecutive Shifts the store is $(0, [a, I(a, b), 0; w], 0, 0)$. The first Shift exposes a in cache. The second either halts if $a = 1$, or gives $(a, [0; w], 0, I(a, b))$. Successful MP then returns $(b, [0; w], b, 0)$, and the total Pop returns $(b, w, 0, 0)$. The zero is inserted once at the compiler level, below the premise computations. Nothing assumes that a numerical zero pair has a distinct list representation.

Lemma 4.1 (All-store refutation compiler)**.** *There is a primitive-recursive map from a $\mathcal{C}_Q$-derivation p to a finite command word $C(p)$ such that, from every $(t, w, 0, 0)$, it either halts or returns* $(\mathrm{concl}(p), w, 0, 0)$*. If p concludes code one, then $A_0; C(p); \mathsf{S}$ halts from every natural store. PA proves these assertions about coded finite executions.*

Proof. Induct on the coded derivation. The axiom cases use the selectors above; the generalization case uses Gen_i, and MP uses (9). Earlier halting remains halting when a suffix is appended. AX first clears an arbitrary offset and cache, so the refutation word works from arbitrary carried stores. If no earlier halt occurs, its last Shift sees $1 + 0 = 1$. The command words are fixed finite templates, and their arithmetic correctness is uniform in the displayed parameters. The recursive proof traversal and finite trace construction are formalizable in PA without using a reflection principle or assuming consistency. □

4.5 Soundness and the fair scheduler

For soundness, annotate the current top, cached formula, and every live canonical stack entry by a $\mathcal{C}_Q$ derivation, or by a PA proof of its arithmetic translation. The offset need not be a theorem code. AX emits valid instances, a generalization of the old cached theorem, and the old top. On MP success, the two theorem annotations give a proof of the consequent; failure gives truth. Shift transports annotations. If its halt test succeeds, one of its two annotated naturals is one. This supplies a refutation. For $w > 0$ the Cantor tail is strictly smaller than w, so canonical stack annotations have a bounded primitive-recursive construction. Numerical stack zero supplies the fixed truth proof.

The source scheduler is unchanged. It enumerates finite command words and carries the store between trials. A single word that works only after a reset would therefore be insufficient; Lemma 4.1 provides the needed uniformity. For an explicit cofinal family, put $E(\epsilon) = 0$ and $E(c :: v) = 1 + \mathrm{encodeCommand}(c, E(v))$. The AX command code is

$$\mathrm{encodeCommand}(\mathsf{AX}_{A,x,y}, z) = p(A + 2, p(y, p(x, z))).$$

If $z = E(A_0; C(p); \mathsf{S})$, fresh candidate codes $p(k + 2, p(0, p(0, z)))$ describe the same universal halting suffix with an arbitrary initial AX. They are strictly increasing and unbounded in k. Starting the fresh counter at two loses none of this family. Decoded positive continuations decrease, and the fixed arithmetic macro loops terminate by their natural counters. PA verifies the resulting finite enumeration and trace construction.

5 Source construction and literal tape verification

5.1 Two changes commute

The builder begins with the attributed PA196 numerical register program and the supplied separate-head PA194 emitter. Its two edits are:

1. replace the 19-token block for f_6 by the equal-length block for f_6^Q, keeping all other 171 macro tokens unchanged;
2. in the successful MP branch, replace source subtree `node55` = (`node49`, `node54`) by `node54`, bypassing the dummy-zero push and retaining the copy.

In native physical labels, the second edit changes $(164, 1) : 1R98$ to $(164, 1) : 1R114$. The source comparison checks the following square directly:

$$\begin{array}{ccc} R_{\mathcal{C},\mathrm{oldMP}} & \xrightarrow{\text{19-site deletion}} & R_{\mathcal{C},N} \\ \downarrow{\scriptstyle f_6 \mapsto f_6^Q} & & \downarrow{\scriptstyle f_6 \mapsto f_6^Q} \\ R_{\mathcal{C}_Q,\mathrm{oldMP}} & \xrightarrow{\text{19-site deletion}} & R_{\mathcal{C}_Q,N}. \end{array}$$

Both upper/lower full sources have 3,673 sites before the deletion and 3,654 afterward. The deletion interval is $[3634, 3653)$; its only external incoming source edge is $3628 \to 3634$. All remaining instructions and destinations agree under the explicit relocation map, on both sides of the square. The axiom exchange changes exactly 79 of the 3,654 instructions relative to PA194N. The changes are disjoint; agreement of a flattened word alone is not used as evidence of preserved structured jump destinations.

The retained copy loop, relative to its entry, is

Site	Instruction	Zero successor	Positive / next
0	decrement register 4	4	1
1	increment register 1		2
2	increment register 6		3
3	jump		0
4	decrement register 6	7	5
5	increment register 4		6
6	jump		4

With register 1 and scratch 6 initially zero, it copies the proposed consequent from register 4 into register 1, restores register 4, and clears scratch 6. This yields the successful branch of (2) without altering the numerical stack.

5.2 Allocation and simultaneous folds

The raw machine has 134 fixed controls and 67 axiom-reader controls. Seven disjoint history-sensitive folds give

$$134 + 67 - 7 = 194.$$

The builder checks all 28 two-cell cases, four for each folded pair, in the final simultaneously folded table. It also checks the incoming rows that justify the historical bit distinguishing the members of each pair. The full source contracts are subsequently checked against that same final table. The generic arithmetic-block rows remain those of the supplied runtime; the independent audit nevertheless rederives their list lemmas.

5.3 What the universal checks establish

The source frame has nine arbitrary natural register values, a header distance $48 + n$ for arbitrary natural n, and an arbitrary finite nonempty list of arbitrary natural spare values. At source address p, the zero gap is $48 + n - |p|$. Every decrement is checked in both cases. The contract includes the whole tape frame and its zero exterior.

The construction checker uses the supplied arithmetic-block specifications. The separate checker imports neither the optimizer nor that block oracle: it derives four generic list-traversal lemmas from the literal rows, resolves the structured source in two independent ways, checks dense/native conjugacy, and verifies every primitive outcome. Its symbolic list rules are finite instances of elementary induction on unary runs and finite lists. The rule source and its assumptions are included in the archive.

Quantity	Observed value
Primitive source sites	3,654
Universal primitive outcomes, each checker	4,755
Construction-checker symbolic operations	1,506,202
Construction-checker maximum per outcome	488
Independent symbolic operations	1,102,319
Independent maximum per outcome	397
Minimum positive-duration constant	3
Generic list lemmas rederived from literal rows	4
Local fold cases	28
Exact blank startup transitions	58,154

Symbolic-operation counts are checker statistics, not literal TM running times. Unlike finite-value sampling, the source contracts quantify over unbounded register values and spare lists. They are still executable mathematical certificates with an exposed trusted base, not new proof-kernel objects.

The exact finite startup reaches source site 25, native control 84, head position -2, logical registers $(2, 0, \ldots, 0)$, header distance 48, and 26 spare blocks each of value one. Both literal simulators compare the entire finite-support tape with the stated frame; the independent simulator uses a set of one-cells, so no finite tape boundary is silently assumed. This verifies initialization only. It is not extrapolation from a long nonhalting experiment.

Lemma 5.1 (PA-formalizable source/tape transport)**.** *The literal blank machine halts if and only if the initialized PA194NQ source scheduler halts. PA proves the equivalence for coded finite runs.*

Proof. The exact finite startup establishes the initial source frame. Each primitive source outcome has a terminating physical realization with the exact successor frame, or the designated source halt. The generic unary and list lemmas prove these contracts uniformly. Concatenate their finite traces to map a source halt to a physical halt. Conversely, let a first physical halt occur at time T. Iterate the totalized source step function $T + 1$ times. If a source halt occurs, reflection is established. Otherwise concatenate the verified positive, nonhalting primitive traces after the nonhalting startup. Their total duration exceeds T, contradicting determinism and the assumed first physical halt. This is a bounded finite construction, not an assumption of an infinite source execution. The integer polynomials, list induction, finite case splits, source recursion, and trace concatenations are all formalizable in PA. No assumption of consistency is used. □

6 Consistency calibration and inherited dependencies

The supplied separate-head calibration gives uniform proof-producing reductions between PA refutations and $\mathcal{C}$-derivations of code one [2, 3]. We retain its detailed texts and distinguish that mathematical dependency from the newly run physical checks. The two directions require different arguments; the upstream exported implication alone is insufficient.

6.1 PA proofs produced from numerical derivations

Translate $M(a, b)$ to finite binary membership $\mathrm{Bit}(x_a, x_b)$, $E(a, b)$ to equality, and $U(a, B)$ to universal quantification of the translated formula. One suitable formula is

$$\mathrm{Bit}(s, z) \equiv \exists u, v\, [s = (2u + 1)2^z + v \ \wedge \ v < 2^z].$$

PA proves $\mathrm{Bit}(s, z) \to z < s$, hence the negation of the translation of code one. Decode zero as truth; the numerical equality $I(0, 0) = 0$ is handled by a fixed truth-equivalence proof.

The substantive axiom supplier is finite binary comprehension. For every arithmetic formula $\theta(z, \vec{u})$, PA proves

$$\forall n\, \exists s < 2^n\, \forall z < n\, (\mathrm{Bit}(s, z) \leftrightarrow \theta(z, \vec{u})).$$

Induct on n, retaining the old code if $\neg\theta(n, \vec{u})$ and adding 2^n otherwise. Substitution into this fixed induction-proof pattern gives a primitive-recursive function producing PA proofs, whose correctness is itself verified in PA. This is a syntactic proof supplier, not a truth predicate for all arithmetic formulas.

Variable-label bounds in formula codes justify the diagonal instance $B \to U(B, B)$. The separation supplier also handles collisions: when the two relevant labels coincide, the binder changes both occurrences, and the first membership antecedent becomes $\mathrm{Bit}(z, z)$. The comprehension matrix must then be changed. The retained proof does not incorrectly hold that bound variable fixed. Other heads have uniform first-order or propositional proof templates. Lemma 3.1 provides the exchanged head's template. Applying these suppliers to the annotation invariant in Section 4 maps a source halt to a PA refutation.

6.2 Numerical derivations produced from PA refutations

The inherited PA196 proof compiler uses two-way membership rewriting, neighborhood equality, and an equality witness. The separate diagonal calculus derives those interfaces without the deleted equality-vacuity head. Its explicit replacements use a formula's own code as a fresh separator label, pointed separation, and fixed classical implication templates. The supplied separate-head documents retain the detailed derivations, including coincident-label cases and the change to a ten-entry axiom interface. These are the logical dependencies behind the forward proof translation; source simulation alone would not establish them.

Compose that PA-to-$\mathcal{C}$ proof translation with Lemma 3.1, then with the all-store compiler Lemma 4.1 and the unchanged fair scheduler. This constructs a finite source halt from each PA refutation. Every transformation is primitive recursive and its correctness is proved by PA induction on the appropriate coded derivation or trace.

Thus, writing H_R for source halting, the two syntactic directions give

$$\mathsf{PA} \vdash H_R \leftrightarrow \exists p\ \mathrm{Prf}_{\mathsf{PA}}(p, \ulcorner 0 = 1 \urcorner).$$

Compose with Lemma 5.1 and negate the existential refutation predicate to obtain Theorem 1.1. The same reasoning also proves, in PA, equality of the blank-halting conditions of PA194NQ and PA194N. It does not prove unconditional nonhalting in PA. In the standard model, nonhalting follows if PA is consistent.

6.3 Dependency ledger

Component	Evidence and scope
Separate-head arithmetic calibration	Retained ordinary proof texts: `original_pa195_calibration.tex`, `separate_head_PA_calibration.tex`, and `diagonal_calculus.tex`. These contain the nontrivial inherited PA/refutation translations.

Component	Evidence and scope
Derived propositional basis	Supplied `ImplicationBasis.lean` and `RestrictedImplication.lean`; supplied 61-line exchange certificate, freshly replayed over their K/S derived interface. No new full Lean build is asserted.
NQ command calibration	Explicit contracts and all-store compiler in this report; free-constructor emitter and symbolic command-word replay in `mathematics/check_commands.py`.
Exact source composition	Builder checks the commuting source square, 19-site deletion, resolved jump targets, 79 exchange differences, and all seven simultaneous folds.
Literal tape refinement	Fresh construction and independent universal checks, four rederived list lemmas, exact whole blank startup, and positive duration certificates.
Novelty	Complete action tuples against all 34 prior frontier classes; source/command examples establish materiality separately.

7 Why the alternative is materially different

Graph novelty by itself is insufficient: one can change unreachable rows or merely renumber states. The comparison therefore has two parts.

First, `comparison/check_novelty.py` verifies the frozen comparison tables' hashes, canonicalizes their complete transition tuples at every graph-spanning root, and compares tuples exactly. It additionally allows left/right reflection and global symbol exchange. There is no match to PA194NQ among any of the 34 classes, including rejected and auxiliary ones. Hashes identify files but are not the sole graph-equality test.

Second, the changed behavior is visible at repeatedly used semantic interfaces. Compared with PA194N, AX emits f_6^Q instead of f_6, with 79 resolved primitive instructions changed. Compared with PA194Q's cached controller, successful MP has a different storage effect. For example, $I(0,2) = 20$, so from $(0,0,0,20)$, MP_2 returns $(2,0,2,0)$ in NQ, whereas Q's cached MP returns $(0,0,0,2)$. These are actual command semantics. NQ retains the original Shift test $t + h = 1$, and its proof compiler reserves the MP zero below both premises. It is therefore a new composition of two prior mechanisms, with explicit interaction and preservation proofs, rather than a new name for an old table. We claim novelty relative to the pinned corpus, not all possible unpublished machines.

8 Attempts below 194 and a rejected 193-state machine

8.1 Protected-runtime quotient search

We extended the existing pair-quotient searches to permit changes to the 23 protected arithmetic-runtime controls. Each candidate removes one control, redirects incoming targets, and selects each survivor row from the corresponding row of either old control. Source-frame control 84 remains fixed. The new search does not assume the altered arithmetic blocks remain valid: it rederives the four generic list lemmas from the candidate's literal rows with normalized entry and exit labels.

Family	Candidates	List lemmas	Representatives	Full source
Protected/protected	1,012	629	0	0
Mixed, protected row changed	11,730	8,334	2	0
Total	12,742	8,963	2	0

The 32 representative contracts cover the primitive instruction/branch kinds. The two surviving mixed candidates fail the complete source check. The cap is 700 symbolic operations per contract; the unchanged HS/NS baseline's recorded maximum is 403. Failure can mean that a required symbolic traversal was not established or that the checker did not close within its bound. Consequently this is a bounded failure to certify a reduction, not a proof that every tested table is semantically invalid and not a lower bound on all PA-consistency machines. The search source, inputs, licenses, and receipts are retained in `search/protected_runtime/`.

A separate fresh search on PA194NQ keeps the 23 protected arithmetic controls fixed and considers every pair among the other 171 controls, with all four old-row selections: $4\binom{171}{2} = 58{,}140$ candidates. It checks the affected

contracts from the full 4,755-outcome collection, with a cap of 1,024 symbolic operations, and finds no survivor. The baseline run establishes which contracts can visit each changed control; unaffected traces cannot encounter a first changed transition. Its source and receipt are `search/search_nq_pairs.py` and `search/nq_pairs_result.json`. This has the same bounded-method limitation, and is not a global lower bound.

8.2 Other investigated changes

A destructive successful-MP routine returning $(b, w, 0, 0)$, consuming the offset instead of copying it, is a distinct source proposal. Its direct implementation currently needs 195 states. The new transfer reader survived the baseline physical checks but no extra fold was found among 684 whole-source row-selection quotients or the tested four-cell, 128-step contextual detours. These bounds do not exclude another implementation. Reusing an AX subtree as an MP epilogue was also explored; additional incoming edges can invalidate the historical-bit assumption of an existing reader fold, so a favorable old decoder score is insufficient.

That cross-command-sharing branch did produce a fully specified 195-state research candidate. Its successful MP returns $(U(b,b),[b,0;w],b,0)$, reusing an existing AX decoder fragment. All 4,886 source outcomes, 32 simultaneous fold cases and a 58,168-step whole startup pass the construction checker. Its logical calibration is supplied as an ordinary proof outline; no new independent literal-tape replay or completed consistency theorem is asserted for this supplementary candidate. Seven completed searches recorded 14,947,566 mutation events, including revisits, without an accepted 194-state reduction in this branch. `search/cross_command/` preserves the small portable research package, its scoped ledger, and a positive truth-sentinel proposal. None of these event counts is a count of distinct machines or a state-count lower bound.

A reproduced zero/positive abstraction of the primary source reaches 263,888 source/mask pairs and certifies no pair of the nine logical registers as directly coalescible. This confirms an already supplied negative result; it is not claimed as a new register-allocation theorem. More precise invariants or a different data representation remain possible.

8.3 A literal 193-state shortcut with the wrong logic

The coefficient-three multiplier is physically smaller. Replace the four-increment constructor body by three increments, using $\texttt{node61} = (\mathrm{inc}_1, \texttt{node60})$. In the native table, the new multiplier rows are $(190, 0) : 0R61$ and $(190, 1) : 1R99$. The additional history fold identifies control 41 with 99 without adding the helper required by the coefficient-five/six constructions. The count becomes

$$134 + 67 - 8 = 193.$$

This rejected machine has 3,612 source sites and 4,719 primitive outcomes. The construction checker passes all outcomes and all 32 simultaneous fold cases; its exact startup reaches the clean source seed in 58,320 steps. Its dense table hash begins `dedb74819ee5`. These physical successes do not establish the desired logical calibration.

8.4 A concrete obstruction to the three-residue shortcut

The tempting change from coefficient four to coefficient three is not merely unsupported by the old decoding proof. With the unchanged fifth head it makes the numerical calculus inconsistent. Thus a new interpretation of those same numerical axioms cannot repair this particular shortcut.

Write

$$p(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \quad I(a,b) = 3p(a,b), \quad E(a,b) = 3p(a,b) + 3,$$
$$U(d,A) = 3p(A,d) + 2, \qquad N(A) = I(A,1).$$

Retain modus ponens, generalization from A to $U(d,A)$, the restricted implicational head

$$R(a,b,c,u) = I\big(I(I(c,u),a), I(I(a,c), I(b,c))\big), \qquad u \le a,$$

and the fifth head

$$H_5(d,h) = N\big(U(d, N(E(h,d)))\big).$$

No equality-reflexivity axiom, equality-substitution rule, separation gate, or semantic soundness assumption will be used.

Proposition. The resulting numerical calculus derives its designated contradiction, code 1, by a finite derivation. Consequently any faithful exhaustive proof search for this unchanged calculus halts.

Proof. First, $R(0,0,0,0) = 0$, so code 0 is a theorem. The numerical collision is

$$E(1,0) = 6 = I(0,1).$$

The generic restricted-implication derivation gives the usual derived K and S schemas

$$K(a,b) = I(a, I(b,a)), \qquad S(a,b,c) = I\big(I(a, I(b,c)), I(I(a,b), I(a,c))\big).$$

These derivations still apply at coefficient three. Specifically, `M6.Logic.RestrictedImplication.k` and `M6.Logic.RestrictedImplication.s` are generic over the implication operation and comparison relation. They require only the displayed restricted head, modus ponens, reflexivity of comparison, and propagation of a bound into either argument of implication. On natural numbers with ordinary $\leq$, these bounds follow from

$$a, b \leq p(a,b) \leq 3p(a,b) = I(a,b).$$

Neither injectivity nor disjoint constructor tags is a hypothesis of those generic derivations. The supplied generic Lean source is preserved with its original attribution in the accompanying certificate directory; no new Lean compilation is asserted here.

Put $a = 6 = I(0,1)$. From K and S derive $I(a,a)$ in the standard way: apply $S(a, I(a,a), a)$ to $K(a, I(a,a))$ and then to $K(a,a)$. From $K(0,a)$ and the theorem 0, derive $I(a,0)$. Now apply $S(a,0,1)$ first to $I(a,a) = I(a, I(0,1))$ and then to $I(a,0)$, obtaining

$$I(a,1) = N(6) = 87.$$

Generalization gives

$$U(0, N(6)) = 11486.$$

But $H_5(0,1)$ is precisely its negation:

$$H_5(0,1) = N(U(0, N(E(1,0)))) = I(11486, 1) = 197943987.$$

One final modus ponens derives 1. □

Executable certificates and scope. There are two complementary certificates. First, `check.py` constructs and checks a fourteen-line numeric certificate in `proof.json`. It checks the displayed small code identities and all instantiated K/S formulas using exact integers.

Second, `expand_basis.py` instantiates the retained generic restricted-implication derivations and expands every use of K and S into primitive rules. The resulting `primitive_proof.json` contains 73,393 proof steps: 34,888 restricted-head instances, 38,503 modus-ponens steps, one generalization, and one fifth-head instance. Its conclusions are represented by a shared directed acyclic graph of 116,171 exact arithmetic expressions, with maximum expression depth 36. Sharing avoids expanding very large numerals and repeated arithmetic subexpressions; it does not change their values or the calculus.

The separate `verify_primitive.py` checker reads only that certificate. It neither parses nor trusts the generic Lean source, admits no K/S rule, and uses no propositional-tautology oracle. For each restricted-head side condition it verifies an explicit chain of implication components, each justified by $a, b \leq I(a,b)$. It checks every modus-ponens identity by arithmetic-expression congruence, using only the normalization $I(0,0) = 0$; it checks the one generalization and the precise numeric fifth-head instance. The final conclusion is literal code 1. The final primitive certificate SHA-256 is

`baf316dce45568036fd803382b30ae2669eb504222a39a6be1f3abcad77dcb83.`

This supplies a finite primitive-rule refutation with a small independent checker. It is not presented as a newly compiled proof-assistant theorem or as a literal-machine halting trace.

This rules out the unchanged coefficient-three emitter, including any state saving obtained solely by implementing its three-increment multiplier more compactly. A viable three-residue design would need to change the actual logical interface or emitted heads, and establish a new calibration. It cannot retain R, H_5, generalization and modus ponens exactly as displayed while preserving consistency.

The rejected table and its physical evidence are deliberately kept below a `search/rejected193/` directory. They are not an accepted smaller consistency machine. The correct lesson is that physical refinement and logical calibration are separate obligations: a complete physical implementation of an inconsistent proof calculus is the wrong answer to the requested consistency problem.

9 Reproduction and artifact map

The accepted construction and all principal checks use Python 3.10 or later and its standard library. Run with assertions enabled; do not use `-O` or `PYTHONOPTIMIZE`. No network, repository checkout, solver, or proof assistant is needed to reconstruct the accepted table and replay its packaged certificates.

From the archive's root, run

```
python3 -B reproduce.py
```

This checks the release hashes, reconstructs the machine in a temporary copy, replays both physical checkers, checks command and exchange certificates, and compares the graph against all prior frontier classes. The full symbolic replay takes several minutes. A separately labeled `-quick` mode omits the independent universal tape replay; it is not a substitute for that proof component.

Individual commands are

```
python3 -B construction/build.py
python3 -B verification/check.py
python3 -B mathematics/check_commands.py
python3 -B mathematics/check_exchange_binding.py
python3 -B mathematics/exchange_certificate.py
python3 -B comparison/check_novelty.py machine/machine.tm
python3 -B search/factor3_obstruction/check.py
python3 -B search/factor3_obstruction/verify_primitive.py
```

The individual commands regenerate receipts in place. Use the top-level temporary-copy runner to preserve the release manifest. The complete corpus rescan additionally requires the pinned Git checkout and is described in `comparison/AUDIT.md`; the frozen novelty check is self-contained.

Path	Contents
`report/`	This PDF, its LaTeX source, and complete transition appendix.
`machine/`	Dense/native/raw tables, labels, source graph, source addresses, folds.
`construction/`	Deterministic builder, frozen grammars, upstream source inputs.
`verification/`	Separate literal-tape checker and attributed induction rules.
`mathematics/`	Exact emitter, all-store command words, and premise-exchange checks.
`evidence/`	Fresh observed construction, physical, logical, and novelty receipts.
`comparison/`	All 51 packet entries, all 34 frontier classes, full census and comparator.
`references/`	Inherited ordinary calibration and precise propositional dependencies.
`search/`	Scoped unsuccessful searches and the rejected factor-three shortcut.

Compile twice from `report/` with

```
pdflatex -interaction=nonstopmode -halt-on-error pa194nq.tex
```

Attribution, licenses, source commit, and exact imported paths are retained in `NOTICE.txt`, `PROVENANCE.json`, and the adjacent license files. The original repository is not modified or published by this deliverable.

A Complete accepted transition table

An entry $wD\,q'$ means write bit w, move in direction D, and enter state q'. Label H is halt 194. The four column groups are only a printing layout; together they list all states 0 through 193 and both read-symbol transitions. Start in state 0 on blank tape.

q	read 0	read 1	q	read 0	read 1	q	read 0	read 1	q	read 0	read 1
0	$1R\,1$	$1R\,2$	49	$0R\,69$	$0L\,14$	98	$0R\,87$	$1L\,115$	147	$0R\,155$	$1R\,146$
1	$1R\,3$	$1R\,1$	50	$0R\,70$	$0R\,71$	99	$0R\,104$	$0L\,14$	148	$0R\,156$	$1R\,157$
2	$0R\,0$	$1R\,2$	51	$0R\,72$	$0L\,49$	100	$0R\,36$	$1R\,116$	149	$0R\,158$	$1R\,159$
3	$0R\,4$	$0R\,1$	52	$0R\,73$	$1R\,54$	101	$0R\,115$	$1R\,78$	150	$0L\,159$	$1R\,160$
4	$1R\,5$	$1L\,6$	53	$0L\,74$	$0L\,15$	102	$0L\,46$	$0R\,113$	151	$0R\,41$	$1L\,161$
5	$1L\,7$	$1L\,8$	54	$0R\,75$	$0L\,10$	103	$0L\,45$	$1R\,117$	152	$0R\,162$	$1R\,163$
6	$0L\,7$	$1L\,6$	55	$0R\,76$	$1R\,77$	104	$0L\,94$	$1R\,118$	153	$0R\,164$	$1R\,164$
7	$0L\,9$	$1L\,6$	56	$0R\,78$	$1R\,79$	105	$0R\,119$	$0L\,14$	154	$0R\,165$	$1R\,166$
8	$0L\,10$	$0L\,11$	57	$0R\,80$	$1R\,81$	106	$0R\,71$	$1R\,106$	155	$0R\,96$	$1R\,115$
9	$1R\,12$	$0L\,9$	58	$0R\,82$	$1R\,83$	107	$0R\,120$	$1R\,30$	156	$0R\,79$	$1R\,165$
10	$0L\,9$	$0L\,10$	59	$0L\,84$	$0R\,1$	108	$0L\,46$	$0R\,106$	157	$0R\,167$	$1R\,165$
11	$1L\,8$	$1L\,11$	60	$0R\,85$	$1R\,60$	109	$0R\,70$	$0R\,95$	158	$0R\,168$	$1R\,169$
12	$0R\,12$	$0R\,13$	61	$0L\,76$	$1L\,86$	110	$0R\,59$	$0R\,95$	159	$0R\,170$	$1R\,157$
13	$0L\,14$	$0R\,13$	62	$0R\,87$	$0L\,7$	111	$0L\,65$	$1R\,121$	160	$0R\,135$	$1R\,130$
14	$1L\,15$	$0L\,15$	63	$0R\,88$	$1R\,89$	112	$0R\,122$	$1R\,123$	161	$0R\,138$	$1R\,135$
15	$0L\,15$	$1L\,16$	64	$0R\,42$	$1R\,90$	113	$0R\,14$	$1R\,113$	162	$0R\,171$	$1R\,172$
16	$0L\,17$	$1L\,16$	65	$0R\,91$	$1R\,49$	114	$0R\,68$	$1R\,114$	163	$0R\,173$	$1R\,172$
17	$0L\,18$	$1L\,16$	66	$0R\,92$	$0L\,75$	115	$0R\,124$	$1L\,67$	164	$0L\,174$	$1R\,175$
18	$0L\,19$	$1L\,16$	67	$0R\,86$	$1R\,33$	116	$0R\,125$	$1R\,126$	165	$0L\,162$	$1L\,176$
19	$0L\,20$	$1L\,16$	68	$0R\,93$	$1R\,68$	117	$0R\,120$	$0L\,14$	166	$0R\,161$	$1R\,176$
20	$0L\,21$	$1L\,16$	69	$0L\,94$	$1L\,33$	118	$0R\,94$	$1L\,98$	167	$0R\,100$	$1R\,177$
21	$0L\,22$	$1L\,16$	70	$0L\,84$	$0R\,2$	119	$0R\,127$	$1R\,127$	168	$0R\,134$	$1R\,178$
22	$0L\,23$	$1L\,16$	71	$0R\,95$	$1R\,71$	120	$0R\,70$	$0R\,68$	169	$0R\,150$	$1R\,163$
23	$0R\,23$	$1R\,24$	72	$0R\,75$	$1R\,96$	121	$0R\,128$	$1R\,129$	170	$0R\,179$	$1R\,180$
24	$0R\,25$	$1R\,26$	73	$0R\,59$	$0R\,85$	122	$0R\,32$	$1R\,36$	171	$0R\,167$	$1R\,140$
25	$0R\,27$	$1R\,28$	74	$0R\,86$	$1R\,97$	123	$0R\,31$	$1R\,101$	172	$0R\,134$	$1R\,145$
26	$0R\,29$	$1R\,30$	75	$0R\,42$	$0L\,13$	124	$0R\,59$	$0R\,114$	173	$0R\,181$	$1R\,41$
27	$0R\,31$	$1R\,H$	76	$0R\,87$	$1R\,37$	125	$0R\,102$	$1R\,117$	174	$0R\,175$	$1R\,182$
28	$0L\,32$	$1R\,33$	77	$0R\,30$	$1R\,98$	126	$0R\,82$	$1R\,81$	175	$0R\,167$	$1R\,145$
29	$0R\,34$	$1R\,35$	78	$0R\,42$	$1R\,99$	127	$0R\,120$	$1R\,120$	176	$0R\,183$	$1R\,173$
30	$0L\,36$	$1L\,37$	79	$0R\,100$	$1R\,101$	128	$0R\,130$	$1R\,131$	177	$0R\,91$	$1R\,64$
31	$0R\,38$	$1R\,39$	80	$0R\,102$	$1L\,103$	129	$0R\,132$	$1R\,133$	178	$0R\,184$	$1R\,165$
32	$0R\,40$	$1R\,41$	81	$0R\,30$	$1R\,80$	130	$0R\,134$	$1R\,135$	179	$0R\,134$	$1R\,181$
33	$0R\,42$	$1R\,43$	82	$0R\,97$	$1R\,104$	131	$0R\,136$	$1R\,137$	180	$0R\,185$	$1R\,185$
34	$0R\,44$	$1R\,45$	83	$0R\,30$	$1R\,105$	132	$0R\,138$	$1R\,91$	181	$0R\,167$	$1R\,151$
35	$0R\,46$	$1R\,47$	84	$1R\,84$	$1L\,15$	133	$0R\,139$	$0L\,10$	182	$0R\,151$	$1R\,130$
36	$0R\,48$	$1R\,49$	85	$0R\,106$	$1R\,85$	134	$0R\,100$	$1R\,67$	183	$0R\,164$	$1R\,186$
37	$0R\,50$	$1R\,51$	86	$0R\,73$	$1R\,107$	135	$0R\,140$	$1R\,127$	184	$0R\,140$	$1R\,187$
38	$0R\,52$	$1R\,42$	87	$0L\,46$	$0R\,93$	136	$0R\,141$	$1R\,142$	185	$0R\,188$	$1L\,189$
39	$0R\,53$	$1R\,54$	88	$0R\,108$	$1R\,98$	137	$0R\,143$	$1R\,144$	186	$0R\,190$	$1R\,41$
40	$0R\,55$	$1R\,56$	89	$0R\,108$	$1R\,69$	138	$0R\,145$	$1R\,41$	187	$0R\,32$	$1R\,189$
41	$0R\,57$	$1R\,58$	90	$0R\,109$	$0L\,14$	139	$0R\,146$	$1R\,147$	188	$0R\,120$	$1R\,191$
42	$0R\,59$	$0R\,60$	91	$0R\,110$	$1L\,74$	140	$0L\,148$	$1L\,143$	189	$0R\,148$	$1R\,192$
43	$0R\,61$	$0L\,14$	92	$0R\,111$	$1R\,112$	141	$0R\,28$	$1R\,149$	190	$0R\,41$	$1R\,138$
44	$0R\,62$	$1R\,63$	93	$0R\,113$	$1R\,93$	142	$0R\,135$	$1R\,150$	191	$0R\,174$	$1R\,120$
45	$0R\,55$	$1R\,64$	94	$0R\,50$	$1R\,103$	143	$0R\,151$	$1R\,152$	192	$0R\,182$	$1R\,193$
46	$0L\,65$	$1R\,66$	95	$0R\,114$	$1R\,95$	144	$0R\,153$	$1R\,154$	193	$0R\,161$	$1R\,138$
47	$0R\,36$	$1R\,67$	96	$0R\,30$	$1R\,49$	145	$0L\,130$	$1R\,120$			
48	$0R\,59$	$0R\,68$	97	$0R\,48$	$1R\,61$	146	$0R\,31$	$1R\,53$			

B Pinned primary sources

The following sources are pinned to the comparison commit; the full machine-readable provenance gives individual imported files.

References

[1] V. Reshetnikov and contributors, *ProveIt: PA consistency machines*, commit `3cc7fbf75ba111a0da696ef2f6001dd528891bd5`. https://github.com/VladimirReshetnikov/ProveIt/tree/3cc7fbf75ba111a0da696ef2f6001dd528891bd5/Computability/BusyBeaver/Research/PA.

[2] Supplied *A 194-state Turing machine calibrated to Con(PA)*, `PA/docs/pa194_conpa_2026-11-13/`. The original 67-reader, seven-fold construction and separate-head calibration.

[3] Supplied *PA194N*, `PA/docs/pa194N_conpa_2026-11-13/`. No-push controller, all-store proof compiler, and independent checks.

[4] Supplied *PA194Q and PA194CSZ*, `PA/docs/pa194q_conpa_2026-11-13/`. Exchanged distribution emitter, 61-line certificate, frozen grammar and seven folds.

[5] J. Chang and attributed contributors, imported PA196 development, `PA/jamesyc-upstream/pa196/`. In particular source Program, Conjugacy, ImplicationBasis, and the generic RestrictedImplication results.

[6] Four incoming archives, pinned at the same commit: `pa194hs_conpa_2026-11-13.zip`, `pa194ns_conpa_2026-11-13.zip`, `pa194n5_n6_conpa_2026-11-13.zip`, and `pa194n6_research_2026-11-13.zip`.

[7] ProveIt, `Computability/BusyBeaver/Research/ZFC/` and `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/`, at the same commit. Used for related proof and compiler methods, not as a replacement of the target PA consistency statement.